

Dataframes e outros Objetos

Análise de Dados Financeiros e Econômicos com o R

Marcelo S. Perlin

EA-UFRGS

10/09/2026

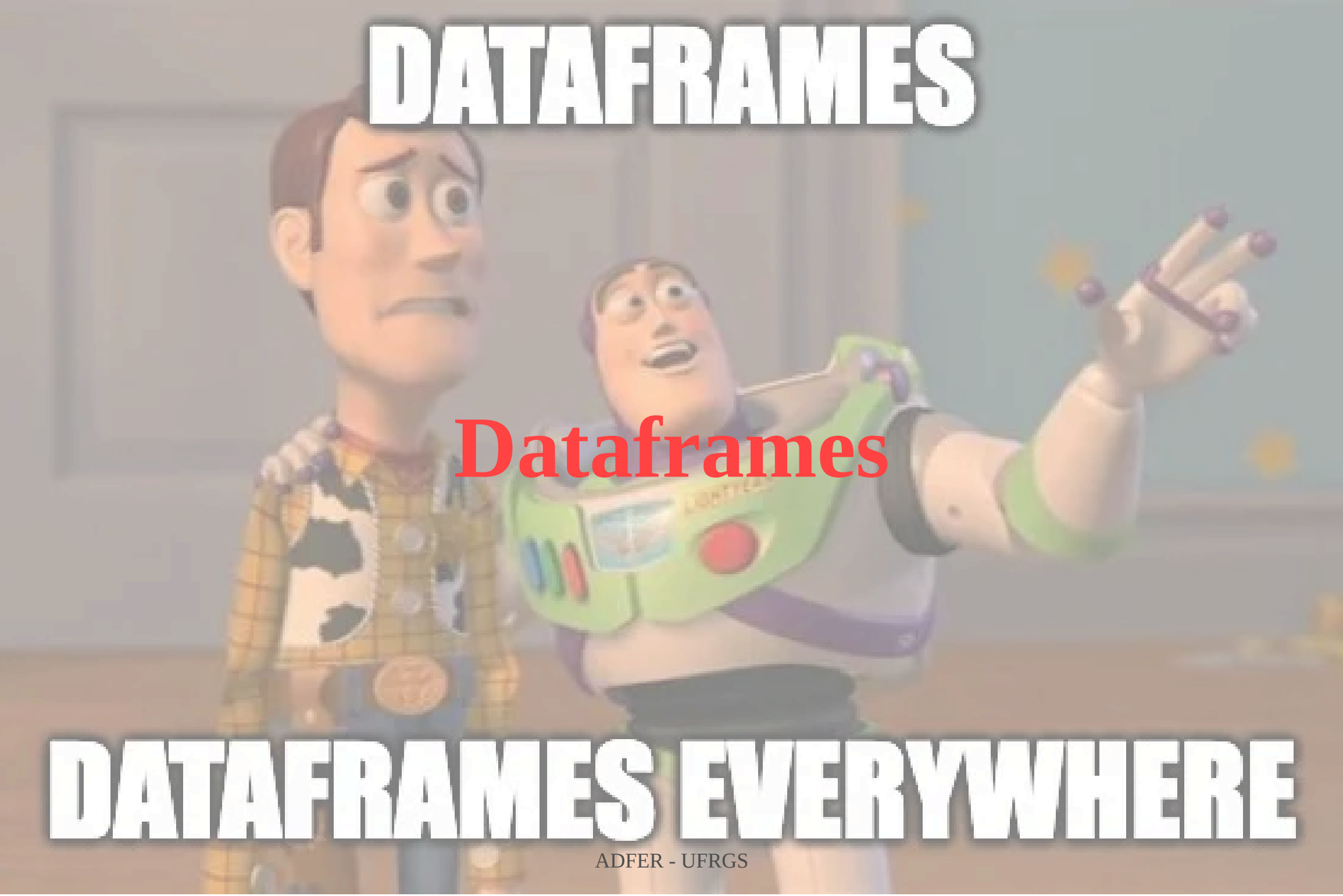
Sumário

- As Classes de Armazenamento no R
- Dataframes
- Operador de *pipeline*
- Listas
- Matrizes
- Tarefa final de aula

As Classes de Armazenamento no R

Introdução

- As classes básicas de objetos no R (números, texto, ...) em vetores são difíceis de trabalhar com dados em larga escala (muitas observações, muitas colunas)
- Classes de armazenamento facilitam a agregação de objetos de forma estruturada
- `dataframes` e `lists` são os objetos mais usados no R

A meme featuring Woody and Buzz Lightyear from the movie Toy Story. Woody is on the left, looking concerned. Buzz is on the right, holding a green blaster and making a peace sign with his fingers. The background is a blurred indoor setting.

DATAFRAMES

Dataframes

DATAFRAMES EVERYWHERE

Introdução

`dataframe` ou `tibble` significa “estrutura ou organização de dados”. Grosso modo, um objeto da classe dataframe nada mais é do que uma **tabela**, com linhas e colunas.

- **principal objeto utilizado no trabalho com o R**, sendo usado como tipo de entrada para diferentes funções
- `dataframe` é um tipo especial de lista, onde cada coluna é um vetor atômico com o mesmo número de elementos, porém com sua própria classe.
- organiza os dados, forçando o pareamento de variáveis
- altamente escalável para dados complexos

Criando dataframes

Exemplo simples

```
1 df <- tibble::tibble(  
2   x = 1:10,  
3   y = sample(letters, 10)  
4 )  
5  
6 dplyr::glimpse(df)
```

Rows: 10

Columns: 2

\$ x <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

\$ y <chr> "l", "j", "p", "v", "e", "w", "c", "o", "q",
"k"

Exemplo menos simples

```
1 # set tickers  
2 ticker <- c(rep('ABEV3', 4),  
3             rep('ADFF4', 4),  
4             rep('FREGS', 2))
```

```

3         rep( 'BBAS3', 4),
4         rep( 'BBDC3', 4))
5
6 # set dates
7 ref_date <- as.Date(rep(c( '2010-01-01', '2010-01-04',
8                           '2010-01-05', '2010-01-06')
9                           3) )
10
11 # set prices
12 price <- c(736.67, 764.14, 768.63, 776.47,
13            59.4   , 59.8   , 59.2   , 59.28,

```

A tibble: 12 × 3

	ticker	ref_date	price
	<chr>	<date>	<dbl>
1	ABEV3	2010-01-01	737.
2	ABEV3	2010-01-04	764.
3	ABEV3	2010-01-05	769.
4	ABEV3	2010-01-06	776.
5	BBAS3	2010-01-01	59.4

Inspecionando um dataframe

Atenção!

Inspecionar tabelas é uma das artes em análise de dados, exigindo intimidade com os dados e o problema em questão.

Estamos interessados em:

- Número de linhas e colunas
- Nomes das colunas
 - sem caracteres latinos, espaços ou até mesmo caixa alta
 - preferencialmente em inglês
- **Classes das colunas**
- **Existência de dados omissos (NA)**
- Verificar se dados **fazem sentido**

Função `dplyr::glimpse()`

```
1 library(dplyr)
2
3 # check content of my_df
4 glimpse(my_df)
```

Rows: 12

Columns: 3

\$ ticker <chr> "ABEV3", "ABEV3", "ABEV3", "ABEV3",
"BBAS3", "BBAS3", "BBAS3"...

\$ ref_date <date> 2010-01-01, 2010-01-04, 2010-01-05,
2010-01-06, 2010-01-01, ...

\$ price <dbl> 736.67, 764.14, 768.63, 776.47, 59.40,
59.80, 59.20, 59.28, 2...

Função `base::summary()`

```
1 # check variation my_df
2 summary(my_df)
```

ticker	ref_date	price
Length :12	Min. :2010-01-01	Min. : 29.81
N.unique : 3	1st Qu.:2010-01-03	1st Qu.: 30.71
N.blank : 0	Median :2010-01-04	Median : 59.34
Min.nchar: 5	Mean :2010-01-04	Mean :283.73
Max.nchar: 5	3rd Qu.:2010-01-05	3rd Qu.:743.54
	Max. :2010-01-06	Max. :776.47

Função `skimr::skim()`

```
1 skimr::skim(my_df)
```

Data summary

Name	my_df
Number of rows	12
Number of columns	3
Column type frequency:	
character	1
Date	1
numeric	1
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	wl
ticker	0	1	5	5	0	3	

Variable type: Date

skim_variable	n_missing	complete_rate	min	max	median	n_unique
ref_date	0	1	2010-01-01	2010-01-06	2010-01-04	4

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p
price	0	1	283.73	353.17	29.81	30.71	59.

Exemplo

 Desafio em aula

Olhe atentamente para a tabela abaixo. Quantos problemas de estrutura ou consistência de dados você consegue identificar antes de avançar?

Data	id.candidato	Nome do candidato	Profissão	age	ano_casamento
10/09/2026	1	Y	professor	8	1999
10/09/2026	1	D	professor	-1	2010
10/09/2026	1	G	professor	9	1995
10/09/2026	1	A	casado	-4	2004
10/09/2026	1	B	advogado	4	2001
10/09/2026	1	K	casado	3	1998
10/09/2026	1	N	advogado	-5	1996
10/09/2026	1	R	advogado	-2	2008
10/09/2026	1	W	advogado	-3	2005
10/09/2026	1	J	advogado	0	1997

Problemas

- coluna `Data` em formato local (dd/mm/yyyy)
- coluna `id.candidato` tem somente valor 1
- coluna `Nome do Candidato` tem espaço no nome da coluna, e valores que não fazem sentido
- coluna `Profissão` tem caractere especial e valor “casado”, o qual não faz sentido
- coluna `age` está em inglês e apresenta **valores negativos** (ex: -5)
- **ninguém com menos de 10 anos pode estar casado!!!!**

Acessando Elementos de um dataframe

- use sempre nomes para acessar colunas (via posição é possível, mas não recomendado)
- toda coluna de um *dataframe* vira um vetor atômico
- todo *dataframe* possui notação matricial para elementos individuais ou blocos

Acessando colunas inteiras

Existem duas principais maneiras de acessar uma coluna de um *dataframe* via nome:

```
1 # isolate columns of df
2 my_ticker <- my_df$ticker
3 my_prices <- my_df[['price']]
4
5 # print contents
6 print(my_ticker)
```

```
[1] "ABEV3" "ABEV3" "ABEV3" "ABEV3" "BBAS3" "BBAS3"
"BBAS3" "BBAS3" "BBDC3"
[10] "BBDC3" "BBDC3" "BBDC3"
```

```
1 print(my_prices)
```

```
[1] 736.67 764.14 768.63 776.47 59.40 59.80 59.20
59.28 29.81 30.82
[11] 30.38 30.20
```

Acessando valores específicos

```
1 # first element
2 my_price <- my_df$price[1]
3
4 print(my_price)
```

```
[1] 736.67
```

```
1 # first 10 elements
2 my_prices <- my_df$price[1:10]
3
4 print(my_prices)
```

```
[1] 736.67 764.14 768.63 776.47 59.40 59.80 59.20
59.28 29.81 30.82
```

```
1 # first 10 rows of first and second column (returns a
2 my_prices <- my_df[1:10, 1:2]
3
4 print(my_prices)
```

```
# A tibble: 10 × 2
  ticker ref_date
  <chr>   <date>
1 ABEV3  2010-01-01
2 ABEV3  2010-01-04
3 ABEV3  2010-01-05
4 ABEV3  2010-01-06
5 BBAS3  2010-01-01
6 BBAS3  2010-01-04
7 BBAS3  2010-01-05
8 BBAS3  2010-01-06
```

Atenção!

Ao usar notação de matriz ([row, column]) em um *dataframe*, o resultado é sempre outro *dataframe*.

An aerial photograph showing a large-scale pipeline project. Multiple parallel pipes, likely made of steel, run from the foreground towards the horizon. The pipes are supported by small structures or spacers. The surrounding terrain is hilly and covered with green vegetation, possibly grass or low shrubs. The perspective is from directly above, looking down the length of the pipeline.

Operador de *pipeline*

Introdução

O operador de *pipeline* é usado com os símbolos `|>` (nativo com R > 4.1) e `%>%` (pacote `magrittr`) e permite que dataframes sejam criados em etapas consecutivas

Imagine o seguinte código, onde a ideia é repassar um *dataframe* entre diferentes e consecutivas funções:

```
1 df1 <- fct1(df0)
2 df2 <- fct2(df1)
3 df3 <- fct3(df2)
```

Uma maneira equivalente e mais elegante:

```
1 my_tab <- my_df |>
2   fct1() |>
3   fct2() |>
4   fct3()
```

Um exemplo prático

```
1 df <- tibble::tibble(  
2   x = 1:10,  
3   y = runif(10)  
4 )  
5  
6 fct1 <- function(df_in) {  
7   df_in$col1 <- 1  
8   return(df_in)  
9 }  
10  
11 fct2 <- function(df_in) {  
12   df_in$col2 <- 2  
13   return(df_in)
```

Rows: 10

Columns: 5

\$ x <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

```
$ y      <dbl> 0.33239467, 0.65087047, 0.25801678,  
0.47854525, 0.76631067, 0.084...  
$ col1 <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1  
$ col2 <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2  
$ col3 <dbl> 3, 3, 3, 3, 3, 3, 3, 3, 3, 3
```

Modificando um dataframe

Para criar novas colunas em um dataframe, basta utilizar a função `dplyr::mutate`

```
1 library(dplyr)
2
3 my_df <- yfR::yf_get(
4   tickers = c('ABEV3.SA', 'PETR4.SA'),
5   first_date = '2023-01-01',
6   last_date = '2023-01-15'
7 )
8
9 # add columns with mutate
10 my_df <- my_df |>
11   group_by(ticker) |>
12   mutate(ret = price_adjusted / dplyr::lag(price_adjus
13   ungroup() |>
```

Rows: 20

Columns: 14

```

$ ticker          <chr> "ABEV3.SA", "ABEV3.SA",
"ABEV3.SA", "ABEV3.SA", ...
$ ref_date        <date> 2023-01-02, 2023-01-03,
2023-01-04, 2023-01-05...
$ price_open      <dbl> 14.40, 14.16, 14.28,
14.27, 14.30, 14.29, 14.26...
$ price_high      <dbl> 14.45, 14.39, 14.47,
14.41, 14.36, 14.47, 14.60...

```

A maneira mais tradicional, e comumente encontrada em código, para criar novas colunas é utilizar o símbolo `$`:

```

1 # add new column with base R
2 my_df$my_seq3 <- 1:nrow(my_df)
3
4 # check it
5 glimpse(my_df)

```

Rows: 20

Columns: 15

```

$ ticker          <chr> "ABEV3.SA", "ABEV3.SA",

```

"ABEV3.SA", "ABEV3.SA", ...

\$ ref_date <date> 2023-01-02, 2023-01-03,
2023-01-04, 2023-01-05...

\$ price_open <dbl> 14.40, 14.16, 14.28,
14.27, 14.30, 14.29, 14.26...

\$ price_high <dbl> 14.45, 14.39, 14.47,
14.41, 14.36, 14.47, 14.60...

Removendo colunas

Para remover colunas de um `dataframe`, basta usar `dplyr::select` com operador negativo para o nome das colunas indesejadas:

```
1 # removing columns
2 my_df_temp <- my_df |>
3   select(-my_seq1, -my_seq2, -my_seq3) |>
4   glimpse()
```

Rows: 20

Columns: 12

\$ ticker <chr> "ABEV3.SA", "ABEV3.SA",
"ABEV3.SA", "ABEV3.SA",...

\$ ref_date <date> 2023-01-02, 2023-01-03,
2023-01-04, 2023-01-05...

\$ price_open <dbl> 14.40, 14.16, 14.28,
14.27, 14.30, 14.29, 14.26...

\$ price_high <dbl> 14.45, 14.39, 14.47,
14.41, 14.36, 14.47, 14.60...

No uso de funções nativas do R, a maneira tradicional de remover colunas é alocar o valor nulo (**NULL**):

```
1 # set temp df
2 temp_df <- my_df
3
4 # remove cols
5 temp_df$price_adjusted <- NULL
6 temp_df$ref_date <- NULL
7
8 # check it
9 glimpse(temp_df)
```

Rows: 20

Columns: 13

\$ ticker <chr> "ABEV3.SA", "ABEV3.SA",

"ABEV3.SA", "ABEV3.SA", ...

\$ price_open <dbl> 14.40, 14.16, 14.28,

14.27, 14.30, 14.29, 14.26...

\$ price_high <dbl> 14.45, 14.39, 14.47,

14.41, 14.36, 14.47, 14.60...

\$ price_low <dbl> 14.02, 14.11, 14.08,
14.14, 14.16, 14.27, 14.25...

Filtrando linhas de um dataframe

- selecione linhas de uma tabela com `dplyr::filter()`:

```
1 library(dplyr)
2
3 # filter df for single stock
4 df_temp1 <- my_df |>
5   filter(ticker == 'ABEV3.SA') |>
6   glimpse()
```

Rows: 10

Columns: 15

\$ ticker <chr> "ABEV3.SA", "ABEV3.SA",
"ABEV3.SA", "ABEV3.SA", ...

\$ ref_date <date> 2023-01-02, 2023-01-03,
2023-01-04, 2023-01-05...

\$ price_open <dbl> 14.40, 14.16, 14.28,
14.27, 14.30, 14.29, 14.26...

\$ price_high <dbl> 14.45, 14.39, 14.47,
14.41, 14.36, 14.47, 14.60...

A função também aceita mais de uma condição. Veja a seguir onde filtramos os dados para 'ABEV3.SA' em datas posteriores ou iguais a '2023-01-10':

```
1 library(dplyr)
2 # filter df for single stock and date
3 my_df_temp <- my_df |>
4   filter(ticker == 'ABEV3.SA',
5          ref_date >= as.Date('2023-01-10')) |>
6   glimpse()
```

Rows: 4

Columns: 15

\$ ticker <chr> "ABEV3.SA", "ABEV3.SA",
"ABEV3.SA", "ABEV3.SA"
\$ ref_date <date> 2023-01-10, 2023-01-11,
2023-01-12, 2023-01-13
\$ price_open <dbl> 14.26, 14.51, 13.80, 14.00
\$ price_high <dbl> 14.60, 14.79, 14.17, 14.05

\$ price_low	<dbl>	14.25,	13.95,	13.66,	13.77
\$ price_close	<dbl>	14 54	14 29	14 10	13 88

Ordenando um dataframe

No `tidyverse`, a forma de ordenar `dataframes` é pelo uso da função `arrange`. No caso de ordenamento decrescente, encapsulamos o nome das colunas com `desc`:

```
1 library(dplyr)
2
3 # set df
4 my_df <- tibble(col1 = c(4, 1, 2),
5                  col2 = c(1, 1, 3),
6                  col3 = c('a', 'b', 'c'))
7
8 # sort ascending, by col1 and col2
9 my_df <- my_df |>
10   arrange(col1, col2) |>
11   print()
```

```
# A tibble: 3 × 3
  col1  col2 col3
<dbl> <dbl> <chr>
```

1	1	1	b
2	2	3	c
3	4	1	a

```
1 # sort descending, col1 and col2
2 my_df <- my_df |>
3   arrange(desc(col1), desc(col2)) |>
4   print()
```

```
# A tibble: 3 × 3
  col1  col2 col3
<dbl> <dbl> <chr>
1     4     1  a
2     2     3  c
3     1     1  b
```

Combinando e Agregando dataframes

- podemos combinar tabelas por linhas (orientação vertical): `dplyr::bind_rows()`
- podemos combinar tabelas por colunas (orientação horizontal):
`dplyr::bind_cols()`
- **agregar** tabelas de acordo com colunas: `dplyr::inner_join`,
`dplyr::left_join`, `dplyr::right_join`, `dplyr::full_join`

Função `dplyr::bind_rows()`

Name	City	Country
Lenna	San Francisco	US
Malcom	New York	US
Akiko	Tokyo	Japan

+

Name	City	Country
Lenna	San Francisco	US
Thomas	London	UK
Diane	Chicago	US



Name	City	Country
Lenna	San Francisco	US
Malcom	New York	US
Akiko	Tokyo	Japan
Thomas	London	UK
Diane	Chicago	US

Exemplo de uso

```
1 library(dplyr)
2
3 # set dfs
4 my_df_1 <- tibble(col1 = 1:5,
5                   col2 = rep('a', 5))
6
7 my_df_2 <- tibble(col1 = 6:10,
8                   col2 = rep('b', 5),
9                   col3 = rep('c', 5))
10
11 # bind by row
12 my_df <- bind_rows(my_df_1, my_df_2) |>
13   glimpse()
```

Rows: 10

Columns: 3

\$ col1 <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

```
$ col2 <chr> "a", "a", "a", "a", "a", "b", "b", "b", "b",  
"b"  
$ col3 <chr> NA, NA, NA, NA, NA, "c", "c", "c", "c", "c"
```

Função `dplyr::bind_cols()`

bind columns

x

A	B	C
a	t	1
b	u	2
c	v	3

+

y

A	B	D
a	t	3
b	u	2
d	w	1

=

A	B	C	A	B	D
a	t	1	a	t	3
b	u	2	b	u	2
c	v	3	d	w	1

CC BY RStudio

Exemplo de uso

```
1 # set dfs
2 my_df_1 <- tibble(col1 = 1:5, col2 = rep('a', 5))
3 my_df_2 <- tibble(col3 = 6:10, col4 = rep('b', 5))
4
5 # bind by column
6 my_df <- bind_cols(my_df_1, my_df_2) |>
7   glimpse()
```

Rows: 5

Columns: 4

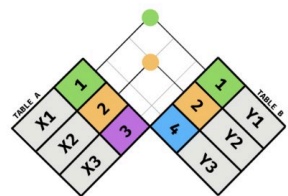
\$ col1 <int> 1, 2, 3, 4, 5

\$ col2 <chr> "a", "a", "a", "a", "a"

\$ col3 <int> 6, 7, 8, 9, 10

\$ col4 <chr> "b", "b", "b", "b", "b"

Funções da família join

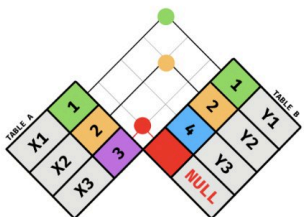


INNER JOIN



```
SELECT
<SELECT LIST>
FROM   TABLE_A A
INNER JOIN TABLE_B B
ON A.KEY = B.KEY
```

KEY	VAL_X	VAL_Y
1	X1	Y1
2	X2	Y2

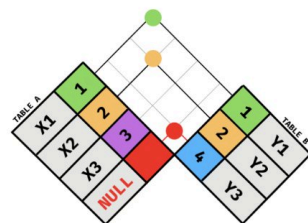


LEFT JOIN



```
SELECT
<SELECT LIST>
FROM   TABLE_A A
LEFT JOIN TABLE_B B
ON A.KEY = B.KEY
```

KEY	VAL_X	VAL_Y
1	X1	Y1
2	X2	Y2
3	X3	NULL

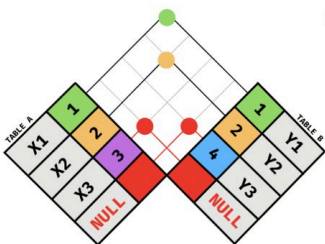


RIGHT JOIN



```
SELECT
<SELECT LIST>
FROM   TABLE_A A
RIGHT JOIN TABLE_B B
ON A.KEY = B.KEY
```

KEY	VAL_X	VAL_Y
1	X1	Y1
2	X2	Y2
4	NULL	Y3



FULL OUTER JOIN



```
SELECT
<SELECT LIST>
FROM   TABLE_A A
FULL OUTER JOIN TABLE_B B
ON A.KEY = B.KEY
```

KEY	VAL_X	VAL_Y
1	X1	Y1
2	X2	Y2
3	X3	NULL
4	NULL	Y3

Image by author, inspired by [R for Data Science](#)

left_join()

Left_join()

Table 1

ID	y
A	5
B	5
C	8
D	0
F	9

Table 2

ID	z
A	30
B	21
C	22
D	25
E	29



Table 3

ID	y	z
A	5	30
B	5	21
C	8	22
D	0	25
F	9	NA

right_join()

right_join()

Table 1

ID	y
A	5
B	5
C	8
D	0
F	9

Table 2

ID	z
A	30
B	21
C	22
D	25
E	29



Table 3

ID	y	z
A	5	30
B	5	21
C	8	22
D	0	25
E	NA	29

inner_join()

inner_join()

Table 1

ID	y
A	5
B	5
C	8
D	0
F	9

Table 2

ID	z
A	30
B	21
C	22
D	25
E	29

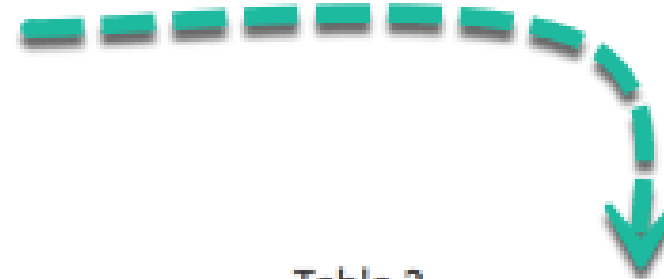


Table 3

ID	y	z
A	5	30
B	5	21
C	8	22
D	0	25

Exemplo `dplyr::inner_join()`

```
1 # set df
2 my_df_1 <- tibble(date = as.Date('2016-01-01')+0:10,
3                   x = 1:11)
4
5 my_df_2 <- tibble(date = as.Date('2016-01-05')+0:10,
6                   y = seq(20,30, length.out = 11))
7
8 # aggregate tables
9 my_df <- inner_join(my_df_1, my_df_2)
10
11 glimpse(my_df)
```

Rows: 7

Columns: 3

\$ date <date> 2016-01-05, 2016-01-06, 2016-01-07, 2016-01-08, 2016-01-09, 2016...

Exemplo `dplyr::left_join()`

```
1 # left_join mantém todas as linhas da tabela à esquerda
2 my_df_left <- left_join(my_df_1, my_df_2, by = "date")
3
4 print(my_df_left, n = 8)
```

```
# A tibble: 11 × 3
```

	date	x	y
	<date>	<int>	<dbl>
1	2016-01-01	1	NA
2	2016-01-02	2	NA
3	2016-01-03	3	NA
4	2016-01-04	4	NA
5	2016-01-05	5	20
6	2016-01-06	6	21
7	2016-01-07	7	22
8	2016-01-08	8	23

Listas

Introdução

Uma lista (`list`) é uma classe de objeto extremamente flexível. Ao contrário de vetores atômicos, a lista não apresenta restrição alguma em relação aos tipos de elementos nela contidos.

- um dos objetos mais utilizados no R
- acomoda informações complexas de forma fácil

Criando Listas

Uma lista pode ser criada através do comando `base::list`, seguido por seus elementos separados por vírgula:

```
1 library(dplyr)
2
3 # create list
4 my_l <- list(c(1, 2, 3),
5              c('a', 'b'),
6              factor(c('A', 'B', 'C')),
7              tibble(col1 = 1:5))
8
9 # use base::print
10 print(my_l)
```

```
[[1]]
```

```
[1] 1 2 3
```

```
[[2]]
```

```
[1] "a" "b"
```

```
[[3]]
```

```
[1] A B C
```

```
Levels: A B C
```

```
1 # use dplyr::glimpse
2 glimpse(my_l)
```

List of 4

```
$ : num [1:3] 1 2 3
```

```
$ : chr [1:2] "a" "b"
```

```
$ : Factor w/ 3 levels "A","B","C": 1 2 3
```

```
$ : tibble [5 × 1] (S3: tbl_df/tbl/data.frame)
```

```
..$ col1: int [1:5] 1 2 3 4 5
```

Também podemos nomear os elementos:

```
1 # set named list
2 my_named_l <- list(ticker = 'TICK4',
3                    market = 'Bovespa',
```

```
4           df_prices = tibble(P = c(1,1.5,2,2.3),
5                               ref_date = Sys.Date())
6
7 # check content
8 glimpse(my_named_l)
```

List of 3

```
$ ticker      : chr "TICK4"
$ market      : chr "Bovespa"
$ df_prices: tibble [4 × 2] (S3: tbl_df/tbl/data.frame)
  ..$ P          : num [1:4] 1 1.5 2 2.3
  ..$ ref_date: Date[1:4], format: "2026-09-10" "2026-09-11" ...
```

Acessando os Elementos de uma Lista

Os elementos de uma lista podem ser acessados através do uso de duplo colchete (`[[`
`]]`) ou `$`, tal como em:

```
1 # accessing elements from list
2 print(my_named_l[[2]]) # using position
```

```
[1] "Bovespa"
```

```
1 print(my_named_l[["df_prices"]]) # using name
```

```
# A tibble: 4 × 2
```

```
  P ref_date
```

```
<dbl> <date>
```

```
1     1  2026-09-10
```

```
2    1.5 2026-09-11
```

```
3     2  2026-09-12
```

```
4    2.3 2026-09-13
```

```
1 print(my_named_l$ticker) # using $
```

Adicionando e Removendo Elementos de uma Lista

Para adicionar ou substituir, basta definir um novo objeto na posição desejada da lista:

```
1 # set list
2 my_l <- list(slot1 = 'a',
3              slot2 = 1:10)
4
5 # add new elements to list
6 my_l$slot3 <- c(1:5)
7 my_l$slot4 <- rep("b", 15)
8
9 # print result
10 glimpse(my_l)
```

List of 4

\$ slot1: chr "a"

Removendo elementos de uma lista

Para remover elementos de uma lista, basta definir o elemento para o símbolo reservado `NULL` (nulo):

```
1 # set list
2 my_l <- list(text = 'b',
3             num1 = 2,
4             num2 = 4)
5
6 # remove elements
7 my_l$num1 <- NULL
8 my_l$num2 <- NULL
9
10 glimpse(my_l)
```

List of 1

\$ text: chr "b"

Processando os Elementos de uma Lista

Toda lista pode ser processada de forma programática através de funções específicas.

- processar elemento por elemento de uma lista

Por exemplo, imagine uma lista com vetores numéricos de diferentes tamanhos, tal como a seguir:

```
1 # set list
2 my_l_num <- list(c(1, 2, 3),
3                 1:50,
4                 seq(-5, 5, by = 0.5))
```

Caso quiséssemos calcular a média de cada elemento de `my_l_num` e apresentar o resultado na tela como um vetor, poderíamos fazer isso através de um procedimento simples, processando cada elemento individualmente:

```
1 # calculate mean of vectors
2 mean_1 <- mean(my_l_num[[1]])
3 mean_2 <- mean(my_l_num[[2]])
4 mean_3 <- mean(my_l_num[[3]])
5
6 # print it
7 print(c(mean_1, mean_2, mean_3))
```

```
[1] 2.0 25.5 0.0
```

O código anterior funciona, porém não é recomendado devido à sua falta de escalabilidade.

Uma maneira mais fácil, elegante e inteligente seria utilizar a função `sapply` (Base R) ou `purrr::map_dbl` (Tidyverse):

```
1 # using base R
2 my_mean <- sapply(my_l_num, mean)
3 print(my_mean)
```

```
[1] 2.0 25.5 0.0
```

Matrizes

Introdução

Uma matriz é uma representação bidimensional de diversos valores, arranjados em linhas e colunas. O uso de matrizes é uma poderosa maneira de representar dados numéricos em duas dimensões e, em certos casos, funções matriciais podem simplificar operações matemáticas complexas.

Um claro exemplo do uso de matrizes em Finanças seria a representação dos preços de diferentes ações ao longo do tempo.

Data	ABEV3	BBAS3	BBDC3
2010-01-01	736.67	59.4	29.81
2010-01-04	764.14	59.8	30.82
2010-01-05	768.63	59.2	30.38
2010-01-06	776.47	59.28	30.20

A matriz anterior poderia ser criada da seguinte forma no R:

```

1 # create matrix
2 data <- c(736.67, 764.14, 768.63, 776.47,
3          59.4, 59.8, 59.2, 59.28,
4          29.81, 30.82, 30.38, 30.20)
5
6 my_mat <- matrix(data, nrow = 4, ncol = 3)
7
8 # set names of cols and rows
9 colnames(my_mat) <- c('ABEV3', 'BBAS3', 'BBDC3')
10 rownames(my_mat) <- c('2010-01-01', '2010-01-04',
11                       '2010-01-05', '2010-01-06')
12
13 print(my_mat)

```

	ABEV3	BBAS3	BBDC3
2010-01-01	736.67	59.40	29.81
2010-01-04	764.14	59.80	30.82
2010-01-05	768.63	59.20	30.38
2010-01-06	776.47	59.28	30.20

Selecionando Valores de uma Matriz

- Os elementos de uma matriz podem ser acessados pela notação `[i, j]`, onde *i* representa a linha e *j* a coluna. Veja o exemplo a seguir:

```
1 my_mat <- matrix(1:9, nrow = 3)
2 print(my_mat)
```

	[, 1]	[, 2]	[, 3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
1 print(my_mat[1, 2])
```

```
[1] 4
```

O uso de testes lógicos para selecionar valores de matrizes também é possível. Veja a seguir:

```
1 my_mat <- matrix(1:9, nrow = 3)
```

```
2 print(my_mat >5)
```

	[,1]	[,2]	[,3]
[1,]	FALSE	FALSE	TRUE
[2,]	FALSE	FALSE	TRUE
[3,]	FALSE	TRUE	TRUE

Tarefa final de aula

Tarefa 03 - Manipulação Básica

1. Crie um dataframe chamado `my_df` com uma coluna chamada `x` contendo uma sequência de 1 a 100 e outra coluna chamada `y` com o valor da coluna `x` adicionada de 5. Usando código, qual a quantidade de valores em `x` maiores que 10 e menores que 25?
2. Use a função `dplyr::mutate` para criar uma nova coluna em `my_df` chamada `cumsum_x`, contendo a soma cumulativa de `x` (função `base::cumsum()`). Desta nova coluna, quantos valores são maiores que 50?
3. Com a função `dplyr::filter()`, filtre o dataframe `my_df` para manter apenas as linhas onde o valor da coluna `x` é maior que 25.

Tarefa 03 - Dados Financeiros e Listas

4. Utilize o pacote `yfR` para baixar dados da ação da META/Facebook (META) desde a data `'2010-01-01'` até hoje. Filtre os dados para manter apenas dados onde o dia da semana era segunda-feira.
5. Importe os dados disponíveis no arquivo `CH11_grunfeld.csv`, disponível no arquivo com os dados do livro. Utilize funções `dplyr::glimpse`, `summary` e `skimr::skim` para conhecer os dados importados.
6. Crie um objeto do tipo lista com três dataframes em seu conteúdo, `df1`, `df2` e `df3`. O conteúdo e tamanho dos dataframes é livre. Utilize a função `sapply` para descobrir o número de linhas e colunas em cada dataframe.

Tarefa 03 - Integração de Mercado

- 7. Usando o pacote `yfR`, baixe os dados da Petrobras (PETR4.SA) e do índice Ibovespa (^BVSP) em dois dataframes diferentes (duas chamadas da função `yfR::yf_get`). Empilhe os dados das duas ações com o comando `dplyr::bind_rows`.
- 8. Usando os mesmos dados do exercício anterior, crie uma nova coluna na tabela da Petrobras com os retornos do índice Ibovespa. A tabela resultante deve ser como a apresentada a seguir:

ref_date	ret_petr4	ret_ibov
2026-09-11	100.7773	100.2455
2026-09-12	101.7379	100.3888
2026-09-13	102.1726	100.6284
2026-09-14	102.8851	100.6874
2026-09-15	103.2851	101.3296
2026-09-16	103.6105	102.2059
2026-09-17	104.3675	102.9848
2026-09-18	104.5702	103.7821
2026-09-19	105.2814	104.2374

ref_date	ret_petr4	ret_ibov
2026-09-20	105.4031	104.6475

Note que você precisará:

- renomear e selecionar colunas da tabela da Petrobras
- renomear e selecionar colunas da tabela do Ibovespa
- realizar o “merge” das tabelas usando as datas como referências

Referências